

MVP2 — The Complete Build Guide

For Founders Who Want to Ship Fast

Agents explained. Claude Code guide. Solo build path.
What to build yourself vs what to hand to a developer.

FOR: Lekha — Founder, Nexpert.ai

VERSION 3.0 // JUNE 2024

CONFIDENTIAL

Section	Topic	Page Focus
01	The Honest Reality Check	What MVP2 actually is vs what got built
02	The 6 AI Agents Explained	What each agent does, why it exists, what the code looks like
03	The Trust Score Algorithm	The math, the weights, what never changes
04	No-Agent Version (Simpler)	How to build MVP2 without agents at all
05	Build It Yourself: Claude Code	Step-by-step guide — what you can build solo in 2 weeks
06	What to Hand to a Developer	Only what you cannot build yourself
07	The 5-Day Sprint Plan	Day by day — what you build, what you verify
08	Hiring an IIT Engineer	What to test, what to pay, what to never outsource
09	The Anti-Checklist	What to avoid — decisions that will waste 3 months

SECTION 01

The Honest Reality Check

Read this before writing a single line of code.

What your developer actually built (MVP1)

A session booking marketplace identical to Topmate and Calendly. Coaches list profiles, learners book sessions, star ratings display. The codebase is well-built PHP — auth, KYC, Razorpay payments, Zoom, admin panel. But there is zero trust infrastructure. No behavioral signals. No Trust Score. No ECI. The entire MVP2 differentiation does not exist yet in any file.

What MVP2 actually is (one sentence)

Every session completed on the platform automatically writes behavioral events to a database table. A deterministic formula reads those events and produces a Trust Score (0-100) for each expert. That score replaces star ratings everywhere a learner sees an expert.

The good news: you are closer than you think

The booking infrastructure, auth system, KYC, payments, and Zoom are all built and working. MVP2 is 3 database tables + 1 PHP file (eci_engine.php) + 6 signal hooks in existing files + UI changes to show the score. The core of MVP2 is approximately 200 lines of new code on top of what already exists.

WHAT IS CHANGING (MVP2 adds)	WHAT IS NOT CHANGING (keep as-is)
3 new database tables (eci_signals, eci_scores, ALTER experts)	All existing DB tables (users, bookings, payments, reviews)
1 new PHP file: includes/eci_engine.php (~60 lines)	The entire PHP booking, auth, KYC, and payment flow
6 signal hooks in existing files (4 lines each)	Zoom integration — just add 1 signal hook at end
Trust Score ring SVG on expert dashboard + browse page	All learner pages except browse and expert profile
Messaging changes: home.php, navigation.php, auth page	Admin panel (except adding Trust Engine page)
Filter/sort by Trust Score on browse page	All API files except adding expert_id from session fix

SECTION 02

The 6 AI Agents — Fully Explained

What each agent actually is, what it does, and what code implements it.

IMPORTANT: Most of these are not "AI" — they are deterministic functions

The word "agent" in this context means "a piece of code responsible for one specific job that runs automatically when triggered." Only 1 of the 6 uses actual AI (NLP sentiment analysis). The rest are PHP/Python functions reading from and writing to the database. Do not let any developer charge you for building "AI agents" — these are simple database operations with a formula.

1

Signal Collector Agent

Event Listener

WHAT IT DOES

Listens for specific events on the platform (session completed, booking made, feedback submitted, no-show) and writes a record to the `eci_signals` table. It is a function called at the end of existing API endpoints.

WHY IT EXISTS

Without this nothing else can run. Real signals are the only thing that makes Trust Score data meaningful. Every other agent depends on this table having data.

TRIGGER

Triggered by: (a) Zoom webhook when session ends, (b) feedback form submit, (c) booking status change to completed/cancelled/no_show, (d) repeat booking detection.

OUTPUT

One row in `eci_signals`: {expert_id, signal_type, signal_value, recorded_at}

2

Behavior Analysis Agent

Aggregator

WHAT IT DOES

Reads all signals from `eci_signals` for a given expert and groups them by type. Counts occurrences of each signal type and sums their values. Produces a dictionary of aggregated signal data ready for the scoring formula.

WHY IT EXISTS

The raw signals are individual events. You need aggregated counts to compute percentages and ratios. This step separates data collection from data computation.

TRIGGER

Called by the Trust Scoring Agent (Agent 6) every time a new signal arrives. No external trigger — internal function call only.

OUTPUT

Python dict or PHP array: {session_completed: 12, feedback_positive: 8, session_cancelled: 1, ...}

3

Consistency Agent

Time-Series Analyzer

WHAT IT DOES

Looks at the distribution of signals over time — not just totals. Checks if the expert has been consistently active (sessions spread over months) vs bursty (all sessions in one week). Produces a consistency ratio used in the `c_consistency` component.

WHY IT EXISTS

An expert with 10 sessions last week and nothing for 6 months should not score the same as an expert with 2 sessions per month for 5 months. Consistency over time is what makes trust meaningful.

TRIGGER

Called as part of the scoring computation. Also runs a scheduled job (daily cron) to decay scores of experts who have been inactive for 60+ days.

OUTPUT

`c_consistency` score (0.0 to 1.0) and an optional `decay_flag` if inactive for 60+ days.

4

Outcome Agent

Result Verifier

WHAT IT DOES

Reads `outcome_verified` signals — signals written when a learner marks a goal as achieved in their dashboard. Cross-checks that the verified outcome corresponds to a real completed booking. Calculates the outcome verification rate (verified outcomes / total sessions).

WHY IT EXISTS

Anyone can claim outcomes. This agent only counts outcomes that have a matching completed booking and explicit learner confirmation. This is the hardest signal to fake and the most valuable one.

TRIGGER

Runs when a learner marks a goal as achieved (POST to outcome API). Also re-checks on demand if admin suspects fraud.

OUTPUT

Updated `c_prediction_accuracy` component. Also sets `outcome_verification_rate` on the expert record.

5

Verification Agent

Identity Checker

WHAT IT DOES

Reads the KYC verification table (already built — `expert_kyc`). When an admin approves KYC, this agent sets `is_eci_verified=1` on the experts table. Verified experts get a teal checkmark badge visible to learners and receive a small score boost (+2 to accountability component).

WHY IT EXISTS

Trust Score without identity verification is gameable. Any behavioral signal is more credible if we know the expert is a real person whose identity has been confirmed.

TRIGGER

Triggered when admin changes KYC status to approved in the KYC verification panel (admin already built — just add a hook).

OUTPUT

Sets `is_eci_verified=1` on experts table. Triggers one `computeECI()` recompute to update score with verified bonus.

6

Trust Scoring Agent

Formula Engine

WHAT IT DOES

The orchestrator. Called after any new signal arrives. Calls Behavior Analysis Agent, gets the 5 component scores, applies the ECI weights (30/25/20/15/10), produces the final score, assigns a band (Sovereign/Established/Verified/Emerging/Unverified), and saves the result to both `experts.eci_score` and `eci_scores` history table.

WHY IT EXISTS

This is the only agent that writes to the experts table. All other agents feed data into this one. Separating the compute step means you can change the weights formula in one place without touching signal collection.

TRIGGER

Called by: Signal Collector (after every new signal), admin recompute button, daily cron (for decay updates).

OUTPUT

Updated `experts.eci_score`, `experts.eci_band`, `experts.eci_band_color`, `experts.eci_last_computed`. New row in `eci_scores` history.

The honest technical summary of all 6 agents

Agent 1 = 4-line function appended to 6 existing API files. Agent 2 = 15-line PHP array aggregation. Agent 3 = 10-line formula using `COUNT()` SQL queries. Agent 4 = 8-line outcome cross-check function. Agent 5 = 3-line hook in admin KYC approval handler. Agent 6 = the 60-line `computeECI()` function in `eci_engine.php`. Total new code: approximately 100 lines. Any developer who quotes more than 3 days for all 6 agents is overcharging.

SECTION 03

The Trust Score Algorithm

The math that must never change. Never expose weights in frontend or logs.

Component	Weight	Formula	What It Measures
prediction_accuracy	30%	$\max(0, (\text{pos_signals} - \text{neg} \times 1.5) / \text{total})$	Did they produce verified outcomes?
consistency	25%	$\max(0, 1 - (\text{cancelled} \times 2 / \text{total}))$	Analyses cancellations and erratic behavior
domain_depth	20%	$\min(1, \text{completed_sessions} / 50)$	Volume of completed work — saturates at 50 sessions
accountability	15%	$\max(0, 1 - (\text{no_shows} \times 2 / \text{total}) + \text{Reconnections} \times 0.05)$	Resolves ghosting
longevity	10%	$\log_{10}(\text{months_active}) / \log_{10}(120)$	Time on platform — logarithmic, maxes at 10 years

```
# RESULT: float between 0.0 and 100.0

# FINAL SCORE FORMULA (never change weights)
score = (c_pred*0.30 + c_cons*0.25 + c_depth*0.20 + c_acct*0.15 + c_long*0.10) * 100
score = round(min(100.0, max(0.0, score)), 1) # clamp to 0-100
```

Band Name	Score Range	Hex Color	What It Means for the Expert
Sovereign	90-100	#00D4AA	Top 10% — highest consistency and outcomes
Established	75-89	#4A90D9	Proven track record — repeat learners, few cancellations
Verified	60-74	#F5A623	Solid performer — building consistency and depth
Emerging	45-59	#E87040	Active but not enough signals yet (< 10 sessions)
Unverified	0-44	#E05555	Insufficient behavioral data — brand new or low engagement

The No-Agent Version

The simplest possible MVP2: 3 database tables + 6 function calls + 1 UI change

THE MINIMUM IMPLEMENTATION — ALL 6 AGENTS AS 1 PHP FILE

THE 6 SIGNAL HOOKS — WHERE TO ADD THEM IN EXISTING FILES

File	Where to Add	Signal Written
admin-panel/apis/expert/sessions.php	After UPDATE bookings SET status="completed"	session_completed
admin-panel/apis/learner/booking.php	After INSERT INTO bookings (repeat check)	repeat_learner
admin-panel/apis/learner/feedback.php	After INSERT INTO reviews	feedback_positive OR feedback_negative
includes/zoom-api.php	After zoom_create_meeting() returns 201	session_completed (backup)
admin-panel/apis/expert/bookings.php	When status changes to cancelled or no_show	session_cancelled OR session_no_show

admin/admin-kyc-verification.php

When admin approves KYC

Sets is_eci_verified=1 + recompute

SECTION 05

Build It Yourself: The Claude Code Guide

A founder with Python skills can ship a working MVP2 prototype in 10 days using Claude Code.

What is Claude Code?

Claude Code is a command-line tool (terminal app) made by Anthropic that has access to your entire codebase. You type instructions in plain English, it reads your files, writes code, runs tests, and makes changes. It is not a chat interface — it is an autonomous coding agent that works inside your actual project folder.

HOW TO INSTALL CLAUDE CODE (5 MINUTES)

```
# Installing Claude Code – one-time setup

# Step 1: Install Node.js (if not installed)
# macOS: brew install node
# Ubuntu: sudo apt install nodejs npm

# Step 2: Install Claude Code globally
npm install -g @anthropic-ai/claude-code

# Step 3: Navigate to your project folder
cd /path/to/nexpert-python # or nexpert-php

# Step 4: Start Claude Code
claude

# It will ask for your Anthropic API key on first run
# Get key from: console.anthropic.com → API Keys → Create Key
```

What Claude Code can do in your Nexpert project

It reads your entire codebase and understands the structure. You say "add the ECI engine file" and it writes the full `eci_engine.php` or `eci_engine.py`. You say "add a signal hook after the session completes in `zoom-api.php`" and it finds the right line and inserts the code. You say "add a Trust Score card to the expert dashboard" and it modifies the HTML in `expert-dashboard.php`. This is the fastest way to implement MVP2 changes without a developer.

EXACT PROMPTS TO GIVE CLAUDE CODE — COPY THESE WORD FOR WORD

Prompt 1 — Create the ECI engine

Give Claude Code this exact prompt:

Create the file `includes/eci_engine.php`. It should define `ECI_WEIGHTS` constant with `prediction_accuracy=0.30`, `consistency=0.25`, `domain_depth=0.20`, `accountability=0.15`, `longevity=0.10`. Define `ECI_BANDS` array with `Sovereign/90/teal`, `Established/75/blue`, `Verified/60/amber`, `Emerging/45/orange`, `Unverified/0/red`. Write function `computeECI(int $expert_id, PDO $pdo)`: array that reads from `eci_signals` table, aggregates by `signal_type`, computes the 5 component scores using the formulas from my developer brief, applies the weights, assigns a band, saves to `experts` table and `eci_scores` history, and returns the result. Write function `recordSignal(int $expert_id, string $signal_type, float $value, PDO $pdo)` that inserts to `eci_signals` and calls `computeECI`.

Prompt 2 — Add Trust Score to dashboard

Give Claude Code this exact prompt:

Modify `expert/expert-dashboard.php`. Replace the Profile Views stat card with a Trust Score card. The card should query `SELECT eci_score, eci_band, eci_band_color FROM experts WHERE user_id=?` using the session `user_id`. Display an SVG ring with `viewBox="0 0 56 56"`, background circle stroke `rgba(0,0,0,.07)`, foreground circle stroke using the band color, `stroke-dasharray="144.5"`, `stroke-dashoffset` calculated as $144.5 - (\text{score}/100) * 144.5$, rotated `-90` degrees. Show the score number inside the ring. Show the band name below. Add a teal left border to the card. Below the stats grid, add a section called "How Your Trust Score Builds" with 4 tiles: `Session Completed`, `Learner Returns`, `Goal Achieved`, `Consistent Over Time` — each with a `+Score` label in teal font-mono.

Prompt 3 — Add signal hook to session completion

Give Claude Code this exact prompt:

In the file that handles session status updates (likely `admin-panel/apis/expert/sessions.php` or `bookings.php`), after the line that sets booking status to "completed", add these 2 lines: `require_once` the path to `eci_engine.php` if not already included, then call `recordSignal($expert_id, "session_completed", 1.0, $pdo)`. The `expert_id` should come from `$_SESSION["user_id"]` matched against the `experts` table — never from `$_POST` or `$_GET`.

Prompt 4 — Fix the security vulnerability

Give Claude Code this exact prompt:

In every PHP file inside `admin-panel/apis/expert/` directory, find every instance of `$expert_id = $_POST["expert_id"]` or `$expert_id = $_GET["expert_id"]` or `$data["expert_id"]` and replace with: `$stmt = $pdo->prepare("SELECT id FROM experts WHERE user_id = ?"); $stmt->execute([$_SESSION["user_id"]]); $expert_id = $stmt->fetchColumn();` Add a check that if `$expert_id` is false, return `json_encode(["success"=>false,"message"=>"Unauthorized"])` with `http_response_code(403)`.

Prompt 5 — Update browse page filters

Give Claude Code this exact prompt:

In learner/learner-browse-experts.php, find the filter section and replace the Minimum Rating select element (which filters by star rating) with a Min Trust Score select that has options: All Scores (value=""), 60+ Verified (value=60), 75+ Established (value=75), 90+ Sovereign (value=90). Change the sort option with value="rating" to value="trust_score" with label "Trust Score". Update the JavaScript fetch call to pass min_trust_score as the parameter name instead of min_rating. In the expert cards rendered by JavaScript, replace the star rating display with the SVG Trust Score ring — same ring component as the dashboard.

Prompt 6 — Create admin trust engine page

Give Claude Code this exact prompt:

Create admin/admin-trust-engine.php. It should require admin auth. It should show 3 sections: (1) Band Distribution — a table showing how many experts are in each of the 5 bands with counts, queried from `SELECT eci_band, COUNT(*) as count FROM experts GROUP BY eci_band`. (2) All Expert Scores — a table with columns: Expert Name, Category, Trust Score, Band, Last Computed, Signal Count. Sorted by `eci_score DESC`. (3) Recent Signals — last 50 signals from `eci_signals` joined with expert name. Add a "Recompute All" button that calls a POST to `admin-panel/apis/admin/trust-engine.php?action=recompute_all`.

Pro tip: Give Claude Code context first

Before any of the above prompts, start each Claude Code session by saying: "I am building Nexpert.ai, a trust intelligence platform for experts. The codebase is PHP with Tailwind CSS. The Trust Score is publicly called Trust Score. The internal name ECI must never appear in any user-facing file. Please read the current structure before making any changes." This context prevents Claude Code from introducing wrong naming.

SECTION 06

Build the Python Proxy Yourself

A founder-grade prototype you can run in Replit or locally. No developer needed.

What a "proxy" means here

A Python/FastAPI version of nexpertapp.com that you run yourself to test the Trust Score system end-to-end before spending money on a developer to integrate it into the PHP codebase. Build it in Replit, test everything works, then hand it to your developer as a reference implementation.

THE 4-FILE PYTHON MVP — WHAT YOU ACTUALLY NEED

Minimum Python MVP structure

```
nexpert-python/  
  main.py           - FastAPI app: all routes, all API endpoints  
  eci_engine.py     - The Trust Score algorithm (40 lines)  
  database.py       - SQLAlchemy setup + all table models  
  requirements.txt  - fastapi uvicorn sqlalchemy pymysql passlib python-jose
```

```
# That is it. 4 files. Everything else is optional.  
# Run with: uvicorn main:app --reload --port 8000
```

STEP-BY-STEP: BUILD IN REPLIT WITH CLAUDE CODE IN 4 HOURS

1

Create a Replit (Python)

Go to replit.com → Create Repl → Python template. Name it nexpert-mvp2-proxy.

- Click + Create Repl → choose Python
- Name it nexpert-mvp2-proxy
- This gives you a Python environment with a public URL immediately

2

Install dependencies

In the Replit Shell tab, run one command.

- `pip install fastapi uvicorn sqlalchemy pymysql passlib[bcrypt] python-jose[cryptography] python-multipart aiofiles jinja2`
- Add to a requirements.txt file so Replit auto-installs on rerun

3

Set up database

Use Replit's built-in PostgreSQL (free) or connect to your existing MySQL.

- In Replit: Tools → Database → enable PostgreSQL (free, no config needed)
- Or: go to Secrets tab → add DB_HOST, DB_NAME, DB_USER, DB_PASS from your Hostinger MySQL

4

Create main.py with Claude Code

Open Claude Code in the Replit terminal and give it the build prompt.

- In Replit Shell: `npm install -g @anthropic-ai/claude-code` then `claude`
- Give it the complete build prompt (see Section 05 for exact wording)
- Claude Code will create `main.py` with all routes and `eci_engine.py` with the algorithm

5

Test the Trust Score endpoint

Verify the core algorithm works before touching any UI.

- Go to `your-replit-url.dev/docs` (FastAPI auto-generates Swagger UI)
- POST to `/api/expert/signal` with `{expert_id: 1, signal_type: "session_completed"}`
- GET `/api/expert/trust-score/1` — should return a score, band, and color

The single Claude Code prompt to build the entire Python proxy

Give Claude Code this prompt to build all 4 files at once: "Build a FastAPI application for Nexpert.ai, a trust intelligence platform for experts. Create: (1) `eci_engine.py` with the `computeECI` function using weights `prediction_accuracy=0.30`, `consistency=0.25`, `domain_depth=0.20`, `accountability=0.15`, `longevity=0.10` reading from `eci_signals` table; (2) `database.py` with SQLAlchemy models for `users`, `experts` (with `eci_score/eci_band/eci_band_color` columns), `bookings`, `eci_signals`, `eci_scores` tables; (3) `main.py` with FastAPI routes: POST `/api/auth/expert` (register+login), POST `/api/expert/signal` (record a signal + recompute), GET `/api/expert/trust-score/{id}` (return score data), GET `/api/experts` (list with filters: `category`, `min_trust_score`, `sort_by`), POST `/api/admin/recompute/{id}` (force recompute); (4) `requirements.txt`. Use JWT for auth. Expert ID must always come from the JWT token, never from the request body.

SECTION 07

The 5-Day Solo Sprint Plan

What you build yourself using Claude Code. By Day 5 the Trust Score is live and computing.

DAY 1

Security + Database (2 hours)

Do not skip. Everything else depends on this.

- Fix payment-config.php: remove hardcoded Razorpay keys, add getenv() calls, add keys to Replit Secrets
- Run 3 SQL migrations: ALTER experts (add 6 ECI columns), CREATE eci_signals table, CREATE eci_scores table
- Verify by running: SELECT eci_score FROM experts LIMIT 1 — should return 50.0
- Fix IDOR in all admin-panel/apis/expert/ files: expert_id from \$_SESSION not \$_POST
- Test: log in as expert, call profile API, confirm it only returns YOUR data

DAY 2

Create eci_engine.php + Session Config (1.5 hours)

The core of MVP2. One file.

- Give Claude Code Prompt 1 from Section 05 — creates eci_engine.php
- Add require_once __DIR__ . "/eci_engine.php"; to includes/session-config.php after currency-config
- Test: call computeECI(1, \$pdo) directly in a test.php file, verify it returns a score
- Check experts table: expert 1 should now have eci_score, eci_band, eci_band_color populated
- Delete test.php

DAY 3

Add Signal Hooks (2 hours)

6 hooks in 6 existing files.

- Give Claude Code Prompt 3 from Section 05 — adds session_completed hook to sessions/bookings API
- Manually add feedback hook in feedback API: record feedback_positive or feedback_negative based on rating
- Add repeat_learner hook in booking creation: check if learner has previous completed bookings with this expert
- Test each hook: complete a test booking, check eci_signals table has a new row after each action
- After 3 test signals, verify computeECI runs and eci_score on experts table has changed from 50.0

DAY 4

Trust Score Visible on Frontend (2 hours)

The expert sees it. The learner sees it.

- Give Claude Code Prompt 2 — adds Trust Score ring to expert dashboard
- Give Claude Code Prompt 5 — updates learner browse page to show rings and Trust Score filter
- Update home.php: change headline to "Your expertise deserves a Trust Score." Remove fake numbers
- Update navigation.php: change Expert Login button to "Apply as Expert" with teal color
- Visual check: go to expert dashboard, should see animated SVG ring with your Trust Score

DAY 5

Admin Trust Engine + Final Check (1.5 hours)

Visibility for you as founder.

- Give Claude Code Prompt 6 — creates admin/admin-trust-engine.php
- Add Trust Engine link to admin-sidebar.php after KYC Verification
- Log in as admin, go to Trust Engine page, verify you can see all expert scores and recent signals
- Do full end-to-end test: expert completes session → signal recorded → score updates → learner sees new score
- Share the URL with one real expert and ask them to apply. That is the go-live moment.

SECTION 08

Hiring an IIT / AI Engineer

What to test, what to pay, what to never outsource.

What you actually need vs what you will be sold

Most candidates will come with impressive CV lines — LLMs, RAG, vector databases, fine-tuning. None of that is relevant to MVP2. What you need is someone who can write clean PHP or Python, understands SQL, and can implement a deterministic algorithm in a legacy codebase without breaking anything. The IIT brand matters less than the ability to read existing code and extend it precisely.

THE TEST ASSIGNMENT — GIVE THIS BEFORE ANY INTERVIEW

2-hour take-home test (no payment required, share as PDF)

Give them this task: "Here is a PHP file (session-config.php) and a database schema (experts table). The experts table has columns eci_score FLOAT DEFAULT 50.0, eci_band VARCHAR(20), eci_band_color VARCHAR(10). Write a PHP function called computeECI(int \$expert_id, PDO \$pdo): array. It should (1) query a table called eci_signals for all rows with this expert_id, (2) count session_completed and feedback_positive rows as positive signals, count session_no_show as negative, (3) compute a score as (positive - negative*1.5) / total * 100, clamped to 0-100, (4) assign a band: above 75 = Established, above 50 = Verified, else Unverified, (5) update the experts table with the score and band, (6) return an array with score, band, and band_color." What you are testing: can they read existing code structure, write clean PDO, handle edge cases (zero signals), and return the right data type. If they pass with clean code in 2 hours, hire them.

Role	Rate (Mumbai/Pune/Remote)	Duration	What They Build
Freelance PHP developer (3-5 yrs exp)	~₹15,000-₹30,000 project fee	1-2 weeks	All 6 signal hooks + eci_engine.php integration into existing codebase
Junior AI engineer (IIT fresher)	₹25,000-₹40,000/month	1 month contract	Python proxy + API layer + Trust Score computation + admin dashboard
Senior full-stack (5+ yrs)	₹60,000-₹1,00,000/month	Part-time, 3 months	Production hardening, caching, webhook reliability, enterprise API
Do yourself with Claude Code	~₹8,000/month (API cost)	1-2 weeks	All MVP2 changes except production hardening

WHAT TO NEVER OUTSOURCE TO A DEVELOPER

NEVER OUTSOURCE:

The ECI weights (30/25/20/15/10) — these are your proprietary IP. No developer should know what they are. Give them only the computeECI function signature, not the formula.

NEVER OUTSOURCE:

The product vision — what the Trust Score means, how bands work, what earns vs deducts signals. Decisions about what behavioral signals matter are founder decisions.

NEVER OUTSOURCE:

Investor narrative — never let a developer explain the algorithm to investors. The CIBIL analogy and the data moat story come from you, not from the code.

NEVER OUTSOURCE:

The public/internal naming distinction — "Trust Score" is public, "ECI" is internal. A developer who calls it ECI in frontend code has broken your IP protection.

NEVER OUTSOURCE:

The database password and Razorpay keys — obvious, but critical. These must be in environment variables only, never in any file a developer has access to on GitHub.

SECTION 09

The Anti-Checklist

Decisions that will waste 3 months. Avoid every one of these.

DO NOT: Rebuilding the entire codebase from scratch

Your PHP codebase works. Auth, payments, KYC, Zoom — all complete. A developer who suggests rewriting in Next.js, React, or "modern architecture" is proposing 3 months of work before you have a single real expert. Add the trust layer to what exists. Rewrite after you have 50 real experts and real revenue.

DO NOT: Using LangChain or any agent framework for the Trust Score

LangChain adds 20 dependencies, 3x the complexity, and a 100ms latency penalty for something that is a SQL query and a formula. The 6 "agents" in this document are PHP functions. They do not need an orchestration framework. Any developer who proposes LangGraph or CrewAI for computing a weighted average is building complexity to justify their rate.

DO NOT: Waiting for the admin Trust Engine before onboarding experts

You can onboard experts and test the Trust Score with nothing more than direct SQL queries and the expert dashboard. The admin Trust Engine is a nice-to-have. The first 50 expert signups matter infinitely more than a dashboard nobody is using yet.

DO NOT: Displaying fake stats on nexpertapp.com while building MVP2

"5,000+ Global Experts" and "50,000+ Active Learners" are currently on your live site with zero real experts. If a potential investor, enterprise client, or journalist checks this URL, it destroys your credibility. Change it to "Currently onboarding founding cohort" today. This takes 5 minutes.

DO NOT: Paying a developer to add the Trust Score ring before you have real signal data

A Trust Score ring showing 50 (the default for no signals) looks worse than no Trust Score at all. Build the signal collection hooks first. Run 30 test sessions with real experts. Get real scores. Then make the ring visible to learners. The UI is worthless without real data behind it.

DO NOT: Calling it "ECI" or "Expert Credibility Index" in any public file

Every PHP file, every HTML attribute, every API response key that says "eci" instead of "trust_score" is a leak of your internal IP. Use "trust_score" in all client-facing code. ECI only appears inside eci_engine.php as a PHP constant that is never output to the browser.

DO NOT: Building a mobile app before the web product works

Nexpert does not need a mobile app right now. It needs 50 real experts with 3 sessions each to generate real Trust Score data. A mobile app for zero users is 2 months of work with zero return. Build it after the first █10 lakh in revenue.

The single most important thing you can do today

Open nexpertapp.com → edit `home.php` → change "2,000+ learners already matched with 100+ verified experts" to "Currently onboarding our founding cohort of verified experts." That is the only change that cannot wait. Everything else in this document is a week of work. That one change takes 3 minutes and protects your credibility with every investor and enterprise client who visits your site while you are building.